

DRPE Project Defense - Interview Preparation

Core Concept: Signature Matching & Mapping

Q: On what exact basis do you detect a risk signature, classify it as a risk, and prioritize it over others?

A: Our platform, DRPE, evaluates and prioritizes risks using a multi-layered approach merging Local Vulnerability Telemetry with Global Threat Intelligence.

1. Local Signature Detection: We use OpenVAS (Greenbone Vulnerability Management) as our local scanner. OpenVAS uses Network Vulnerability Tests (NVTs) containing signatures for known flaws, misconfigurations, or outdated software. When OpenVAS scans a network, it matches these signatures and assigns a base CVSS (Common Vulnerability Scoring System) score, giving us the intrinsic severity.
2. Global Threat Intelligence Context: A high CVSS score isn't enough if no hackers are actively exploiting it. We cross-reference local findings with AlienVault OTX and AbuseIPDB. From OTX, we pull 'Pulses' (Indicators of Compromise) to see if the CVEs are associated with active APT groups. From AbuseIPDB, we check if endpoints are communicating with malicious IPs.
3. The Multi-Variable Risk Engine: Our proprietary engine takes the static CVSS score and applies a Threat Intelligence Modifier. The logic is: $\text{Final Risk Score} = (\text{Base CVSS Score}) + (\text{OTX Exploitation Weight}) + (\text{AbuseIPDB Exposure Weight})$. We prioritize based on Exploitability, meaning a vulnerability confirmed by AlienVault to be actively exploited gets boosted to the top of the priority queue.

Q: How exactly are you mapping and matching these signatures locally and globally?

A: We handle signature matching in two distinct phases:

1. Local Matching (How we find the vulnerability): We rely on the OpenVAS scanning engine which uses NVTs written in NASL (Nessus Attack Scripting Language). The engine matches signatures using Banner Grabbing & Version Matching, Configuration Checking (missing headers, default credentials), and Active Probing (safe exploitation).
2. Global Mapping (How we connect it to Threat Intel): Once OpenVAS confirms a local match, our FastAPI backend parses the report and extracts the CVE ID (e.g., CVE-2021-44228). This CVE ID is our primary mapping key. We query AlienVault OTX to ask if there are active Pulses for this exact CVE ID. If there's a match, we map that global threat data directly to the local vulnerability in our PostgreSQL database.

Q: What happens if a vulnerability doesn't have a CVE ID yet (a zero-day)? How do you map it?

A: If OpenVAS detects a misconfiguration or a flaw without an official CVE ID, we rely on the OpenVAS NVT OID (Object Identifier) for local tracking. Because there is no CVE, it won't map to standard CVE threat feeds. However, our system still evaluates the asset's overall exposure by checking if the asset's IP is communicating with known malicious IPs via our AbuseIPDB integration, applying a behavioral threat modifier.

DRPE Project Defense - Interview Preparation

Architecture & Tech Stack

Q: Why did you choose FastAPI over Express.js (Node.js) or Django?

A: FastAPI is built on ASGI (Asynchronous Server Gateway Interface) making it extremely fast. Since our backend needs to perform heavy, asynchronous I/O operations (querying OpenVAS, AlienVault, AbuseIPDB, and Gemini AI), asynchronous execution is critical to prevent bottlenecks. It also natively supports Pydantic, making validation of complex CVE/CVSS JSON payloads highly efficient.

Q: What is the exact role of Google Gemini 1.5 Flash in your project?

A: Traditional scanners output raw CVE descriptions and complex CVSS vectors which are highly technical. We send structured vulnerability data to the Gemini API. Gemini acts as an analytical translator, processing the technical data and generating natural-language executive summaries and step-by-step mitigation policies for administrators.

Q: How are you managing the performance of 2D/3D Force-Directed Graphs on a web browser?

A: Rendering massive network topologies can cause browser lag. We handle this by using WebGL-accelerated rendering libraries in React (like react-force-graph powered by Three.js) which offloads rendering to the GPU. We also ensure the backend paginates the JSON payload so the frontend only renders what is necessary.

Q: What is the benefit of deploying via Docker Compose?

A: Consistency and rapid deployment. By containerizing the React frontend, FastAPI backend, and PostgreSQL database, we eliminate the 'it works on my machine' problem. Docker Compose ensures all microservices spin up with the exact required dependencies and internal networking bridges using a single command.

DRPE Project Defense - Interview Preparation

Database & Backend Data Modeling

Q: You have a RiskScore table storing scores. Why store them instead of calculating them on the fly?

A: Storing the calculated risk metrics provides historical context. By persisting `normalized_score`, `exploit_bonus`, and `criticality_weight`, we can power endpoints like `trend_data` and `posture_snapshot`. Calculating them on the fly for historical trend graphs would be computationally expensive and require fetching historical threat intel, which changes over time.

Q: Why use JSONB for the references field in your Vulnerability model?

A: JSONB in PostgreSQL is perfect for storing semi-structured data. A vulnerability might have one reference link, or twenty different CVE links and vendor advisories. JSONB allows us to store an array of links directly in the vulnerability row while still allowing PostgreSQL to index and query the JSON keys efficiently.

Q: Why are you using UUIDs for primary keys instead of standard integers?

A: UUIDs provide better security and scalability. If we used sequential integers (e.g., `/api/assets/1`), an attacker could easily guess and enumerate all our assets or scans (IDOR attack). UUIDs are practically impossible to guess, adding a layer of security.

DRPE Project Defense - Interview Preparation

Security & Compliance

Q: You mentioned an 'Immutable Activity Logging framework'. How do you guarantee it is immutable?

A: Our framework tracks every action for auditability. To make it immutable, we can implement PostgreSQL database-level triggers that reject any UPDATE or DELETE statements on the logs table. For stricter compliance, we generate cryptographic hashes for log entries (similar to a blockchain ledger), meaning any tampering breaks the hash chain.

Q: How are you securing the API keys for AbuseIPDB, AlienVault, and Gemini?

A: We strictly avoid hardcoding API keys in our source code. We utilize .env (environment variables) files which are passed into the Docker containers at runtime. These files are added to .gitignore so they are never pushed to GitHub.

Q: How does your JWT authentication module work?

A: We use a two-token system: a short-lived Access Token and a longer-lived Refresh Token. The short-lived token minimizes the window of opportunity if the token is stolen (e.g., via XSS). Refresh Tokens can be stored in HttpOnly cookies, making them completely inaccessible to client-side JavaScript.

Q: Is DRPE an active defense system (IPS) or just an evaluation tool?

A: DRPE is primarily an orchestration and evaluation platform. While it dynamically identifies risks and provides AI-generated mitigation steps, it does not actively drop network packets or change firewall rules on its own. However, its output data could theoretically be fed into a SIEM or Firewall via webhooks for automated blocking in the future.